รายงานโครงงาน ระบบจูเกะซ์ร้าน KPP (Yum Jukebox)

ระบบจูเกะซ์สำหรับร้านกาแฟที่ลูกค้าขอเพลงผ่าน QR Code

ทดสอบใช้งานได้ที่ https://yumpakjure.onrender.com/
ที่เก็บซอร์สโค้ด https://github.com/tlemtv098-dotcom/YumparkjureQRsong

ชื่อ-นามสกุล นักศึกษา: นายสุภวัฒน์ จันไพจิตร

รหัสประจำตัวผู้เรียน: 671320101

รายงานโครงงาน

เพื่อการส่งมอบและประเมินผล

กำหนดส่ง 15 ตุลาคม 2569 (15 October 2026)

บทที่ 1 บทนำ

1.1 ความเป็นมาและปัญหาที่ต้องการแก้ไข

ร้านกาแฟและร้านอาหารที่มีโซนนั่งรอ มักใช้ระบบจูเกะซ์หรือเครื่องเล่นเพลงอัตโนมัติเพื่อให้ลูกค้า เลือกฟังเพลงได้เอง แต่เครื่องจูเกะซ์ที่มีขายทั่วไปมักมีข้อจำกัดสำคัญ 3 ข้อ ได้แก่ รายการเพลงถูก ตรึงไว้ ผู้ใช้โทรศัพท์ที่เป็นเครื่องของร้านเองหรือหน้าเครื่องดัง และไม่มีประวัติว่าเพลงใดถูกขอบ่อย ร้านจึงเสียโอกาสที่จะรู้ความนิยมของลูกค้า

ปัญหาที่ต้องการแก้ไขจึงเป็นการหาวิธีที่จะทำให้ลูกค้าในร้านส่งคำขอเพลงได้เองจากโทรศัพท์ของตน โดยไม่ต้องเดินไปที่เคาน์เตอร์ เพื่อให้คิวเพลงถูกจัดลำดับอย่างเป็นระบบ และให้เจ้าของร้านเห็นภาพรวมของความนิยมนั้นผ่านกราฟ โครงการนี้จึงนำเสนอระบบจูเกะซ์ที่ทำงานบนเว็บ ลูกค้าสแกนคิวอาร์โค้ดที่ติด บนโต๊ะ แล้วเปิดหน้าเว็บไซต์เพื่อค้นหาและขอเพลง เพลงที่ขอเข้ามาจะถูกเรียงลงคิวบนหน้าจอเครื่องเล่นในร้าน และเจ้าของร้านเปิดดูสถิติจากแดชบอร์ดได้

1.2 วัตถุประสงค์ของโครงการ

1. ให้ลูกค้าสมัครสมาชิก เข้าสู่ระบบ และออกจากระบบได้
2. กำหนดบทบาทผู้ใช้อย่างน้อย 2 บทบาท และควบคุมสิทธิ์การเข้าถึงแต่ละส่วนจริง
3. สร้างกระบวนการขอเพลงที่ลูกค้าทำเองได้ ตั้งแต่ค้นหาเพลงจนเพลงเข้าคิว
4. จัดการข้อมูลเพลง ผู้ใช้ แนวเพลง และแท็กด้วย Django Model และฐานข้อมูล
5. ให้เจ้าของร้านเห็นภาพรวมความนิยมของเพลงผ่านกราฟบนแดชบอร์ด
6. ให้บริการผ่านอินเทอร์เน็ต เพื่อให้ลูกค้าใช้งานได้จากอุปกรณ์ของตนเอง

1.3 ขอบเขตของระบบ

ระบบทำงานเป็นเว็บแอปแบบ Server-Rendered ของ Django ประกอบด้วยหน้าเว็บสำหรับผู้ใช้ทุกบทบาท และ REST API จำนวน 26 เส้นทางสำหรับให้ส่วนติดต่อกับระบบโดยไม่ต้องโหลดหน้าใหม่ทั้งหน้า ขอบเขตของงานครอบคลุมตั้งแต่การสมัครสมาชิก การค้นหาเพลงจาก YouTube การเพิ่มเพลงเข้าคิว การเล่นเพลงบนหน้าจอ การจัดการเพลย์ลิสต์ การจัดการผู้ใช้ แนวเพลง และแท็กของผู้ดูแลระบบ รวมถึงแดชบอร์ดสรุปสถิติและการส่งออกข้อมูลรูปแบบ CSV

> รายการที่อยู่นอกขอบเขตคือระบบชำระเงิน การส่งคำขอเพลงผ่านอีเมล และการจัดการสิทธิ์แบบตามฟังก์ชันสำหรับผู้ใช้ทุกคน เพราะโครงการนี้เน้นที่การควบคุมสิทธิ์ตามบทบาทซึ่งทำได้ครบและตรวจสอบได้ชัดเจนกว่า

1.4 ความสำคัญและประโยชน์ที่ได้รับ

- ลดเวลาที่ลูกค้าต้องเดินไปที่เคาน์เตอร์เพื่อขอเพลง
- ทำให้คิวเพลงยุติธรรมขึ้น เพราะทุกคำขอถูกบันทึกเป็นลำดับเวลา
- เจ้าของร้านทราบความนิยมของเพลงจากกราฟ โดยไม่ต้องเปิดหูฟังเพลงทั้งหมด

- เก็บประวัติการขอเพลงไว้วิเคราะห์ต่อได้ และเป็นฐานสำหรับการจัดเพลย์ลิสต์ประจำวัน

บทที่ 2 การวิเคราะห์และออกแบบระบบ

2.1 บทบาทผู้ใช้งานและสิทธิการเข้าถึง

ระบบกำหนดผู้ใช้ 3 บทบาท เก็บไว้ในฟิลด์ role ของโมเดล Profile ได้แก่ admin, staff และ customer
การควบคุมสิทธิทำงานด้วยเดคคอเรเตอร์ role_required ในไฟล์ music/views_auth.py ซึ่งจะตรวจว่าผู้ใช้เข้าสู่ระบบแล้วหรือยัง และบทบาทอยู่ในรายการทีอนุญาตหรือไม่
หากไม่ผ่านจะแจ้งข้อความและพากลับไปหน้าเครื่องเล่น ทันที ไม่ใช่แค่ซ่อนปุ่มในหน้าเว็บ

บทบาท	สิทธิที่ได้รับ	สิทธิที่ถูกปฏิเสธ
admin	ดูแล้ปเดอร์ จัดการผู้ใช้ (เพิม แก้ไข ปิดใช้งาน ลบ) จัดการแนวเพลงและแท็ก จัดการคิวเพลง เปิดแดชบอร์ดและกราฟ ส่งออกข้อมูล CSV	ไม่สามารถลบบัญชีของตนเองทีเป็นผู้ดูแลระบบคนสุดท้ายได้
staff	จัดการคิวเพลง ย้ายลำดับเพลง ล้างคิว จัดการเพลย์ลิสต์ เปิดหน้าเมนูของฝ่ายบริหาร	ไม่สามารถสร้าง แก้ไข หรือลบผู้ใช้ และไม่สามารถจัดการแนวเพลงกับแท็กได้
customer	สมัครและเข้าสู่ระบบ ขอเพลง ดูเพลงทีตนสัง ลบเพลงทีตนสัง สร้างและจัดการเพลย์ลิสต์ของตน แก้ไขโปรไฟล์และรหัสผ่าน	ไม่สามารถเข้าถึงหน้าจัดการผู้ใช้ แนวเพลง แท็ก แดชบอร์ด และคิวเพลงรวมของทั้งร้าน

การตรวจสิทธิซ้ำทีฝั่งเซิร์ฟเวอร์เป็นสิงสำคัญ เพราะการซ่อนปุ่มด้วย HTML หรือ JavaScript
ป้องกันได้แค่ระดับหน้าตา ผู้ใช้ทีเข้าถึง URL โดยตรงยังต้องผ่านการตรวจจากเดคคอเรเตอร์นี ชุดทดสอบ tests_nav_customer.py มีกรณีตรวจสอบว่าบทบาท customer เปิด /management/users/ ไม่ได้จริง

2.2 กรณีการใช้งาน (Use Case)

ผู้ใช้	กรณีการใช้งาน	ผลลัพธ์ทีระบบต้องทำ
ลูกค้า	สมัครสมาชิกใหม่	สร้าง User และ Profile บทบาท customer โดยอัตโนมัติ พร้อมเข้าสู่ระบบทันที
ลูกค้า	ค้นหาเพลงทีอยากฟัง	เรียก YouTube Data API v3 แล้วกรองเฉพาะวิดีโอทีฝังได้และเป็นเพลง พร้อมส่งผลเป็น JSON
ลูกค้า	กดขอเพลง	สร้างแถวใน SongQueue พร้อมชือผู้ขอ และคืนสถานะสำเร็จเป็น JSON
ลูกค้า	เล่นเพลงถัดไปในหน้าเครื่องเล่น	เล่นเพลงถัดไปผ่าน YouTube IFrame API แล้วเรียก /api/played/ เพือทำเครืองหมายว่าเล่นแล้ว
ลูกค้า	สร้างเพลย์ลิสต์และเพิมเพลง	สร้าง Playlist และเขียน PlaylistSong พร้อมกำหนด position ตามลำดับทีเพิม
พนักงาน	จัดการคิวเพลงหน้าเครื่องเล่น	ย้ายลำดับเพลง ลบเพลง หรือล้างคิวทั้งหมดผ่านเดคคอเรเตอร์ role_required

ผู้ใช้	กรณีการใช้งาน	ผลลัพธ์ที่ระบบต้องทำ
ผู้ดูแล	เปิดแดชบอร์ด	คำนวณจำนวนผู้ใช้ ผู้ใช้ที่ใช้งานอยู่ จำนวนแนวเพลง และจำนวนเพลง พร้อมส่งข้อมูลกราฟ 4 ชุด
ผู้ดูแล	เพิ่มหรือลบแนวเพลงและแท็ก	ทำ CRUD ผ่านฟอร์มที่ตรวจความถูกต้องของ slug และความยาวชื่อก่อนบันทึก
ผู้ดูแล	ส่งออกข้อมูล	ส่งออกเพลย์ลิสต์และคิวเพลงเป็นไฟล์ CSV ที่ดาวน์โหลดได้ทันที

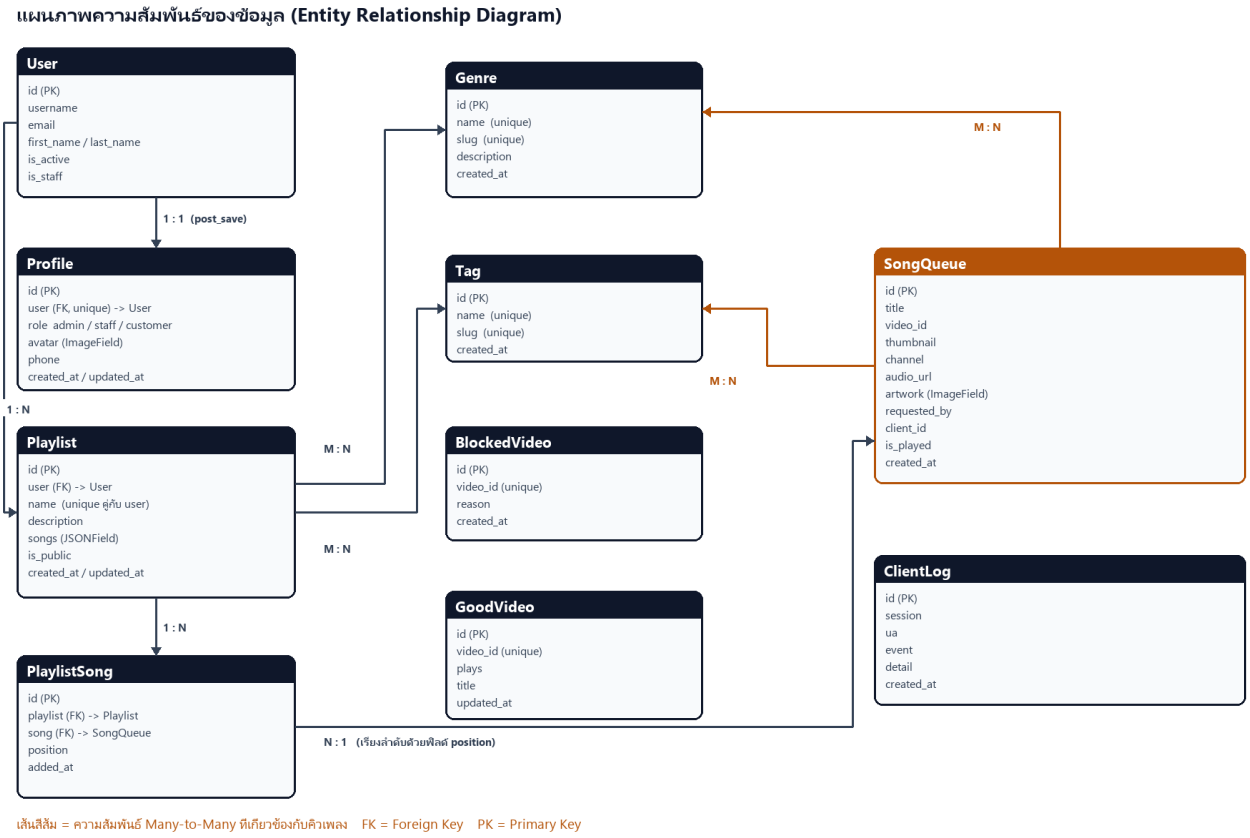
2.3 สถาปัตยกรรมของระบบโดยรวม

ระบบแบ่งเป็น 4 ชั้น ได้แก่ ชั้นนำเสนอ (Template) ที่สร้างหน้า HTML ด้วย Tailwind CSS ชั้นตรรกะ (View) ที่ประมวลผลคำขอและส่งข้อมูลกลับ ชั้นโมเดล (Model) ที่กำหนดโครงสร้างข้อมูลและชุดฟอร์ม และชั้นฐานข้อมูลที่เป็น PostgreSQL (Neon) ในการใช้งานจริงผ่าน DATABASE_URL โดยสลับอัตโนมัติผ่านแพ็กเกจ dj-database-url ส่วนในเครื่องพัฒนาใช้ SQLite

- ผู้ใช้เปิดเว็บไซต์ ระบบส่งหน้าเข้าสู่ระบบและหน้าสมัครสมาชิก
- ผู้ใช้ค้นหาเพลง ระบบเรียก YouTube Data API v3 แล้วกรองผลลัพธ์ฝั่งเซิร์ฟเวอร์
- ผู้ใช้กดขอเพลง ระบบบันทึกลงโมเดล SongQueue และหน้าเครื่องเล่นดึงคิวมาแสดงผล
- หน้าเครื่องเล่นเรียกเพลงถัดไปมาเล่นผ่าน YouTube IFrame API แล้วทำเครื่องหมายว่าเล่นแล้ว
- เจ้าของร้านเปิดแดชบอร์ด ระบบคำนวณสถิติแล้วส่งเป็น JSON ไปยัง Chart.js วาดกราฟ

2.4 แบบจำลองข้อมูล

ระบบมีโมเดล 9 โมเดล และมีความสัมพันธ์ครบทั้งสามแบบที่โจทย์กำหนด ดังภาพ



ภาพที่ 2-1 แผนภาพความสัมพันธ์ของข้อมูล (Entity Relationship Diagram)

2.5 ความสัมพันธ์ระหว่างโมเดลที่ใช้งานจริง

- One-to-One: Profile.user ชี้ไปยัง User ด้วย ForeignKey ที่มีค่า unique แถวของ Profile ถูกสร้างอัตโนมัติโดย signal ที่รับ post_save ของ User เพื่อไม่ให้ผู้ใช้ทุกคนต้องไปกรอกข้อมูลโปรไฟล์เอง
- One-to-Many: Playlist.user ชี้ไปยัง User หนึ่งผู้ใช้มีเพลย์ลิสต์ได้หลายชุด และ SongQueue เก็บ requested_by ไว้ว่าเพลงนั้นถูกสั่งโดยใคร
- Many-to-Many: Playlist.genres, Playlist.tags, SongQueue.genres และ SongQueue.tags ชี้ไปยัง Genre และ Tag ทำให้เพลงหนึ่งเพลงจัดหมวดหมู่ได้หลายหมวด และหมวดหมู่หนึ่งอาจรวมหลายเพลง
- Many-to-Many ผ่านตารางกลาง: ความสัมพันธ์ระหว่าง Playlist กับ SongQueue ใช้โมเดล PlaylistSong เป็นตารางกลาง ซึ่งเก็บฟิลด์ position เพื่อจำลำดับเพลงในเพลย์ลิสต์ วิธีนี้ต่างจากการใช้ ManyToManyField ตรง ๆ ตรงที่เก็บข้อมูลเพิ่มเติมของความสัมพันธะได้ เช่น ลำดับและเวลาที่เพิ่ม

ข้อสังเกตที่ควรรู้: Playlist ยังมีฟิลด์ songs แบบ JSONField เก็บรายการเป็น JSON ไว้ด้วย เพื่อรองรับโค้ดเดิมที่อ่านค่านี้ ส่วนความสัมพันธ์จริงที่ใช้งานอยู่คือตาราง PlaylistSong ฟังก์ชัน add_song เขียนทั้งสองส่วนให้ตรงกันเสมอ

2.6 ขั้นตอนการทำงานของการขอเพลง



ภาพที่ 2-2 ขั้นตอนการทำงานตั้งแต่ค้นหาจนเพลงเข้าคิว

บทที่ 3 การพัฒนาระบบ

3.1 สภาพแวดล้อมการพัฒนา

รายการ	ค่าที่ใช้
ภาษาหลักของเฟรมเวิร์ก	Python 3.12 (Dockerfile ใช้ python:3.12-slim)
เฟรมเวิร์กเว็บ	Django 6.0.6
เว็บเซิร์ฟเวอร์	Gunicorn 21.2.0 (2 workers)
แพ็กเกจประกอบ	yt-dlp, qrcode, pillow, whitenoise, dj-database-url, psycopg
ฐานข้อมูล	PostgreSQL (Neon) ในการใช้งานจริงผ่าน DATABASE_URL และ SQLite (db.sqlite3) สำหรับพัฒนาในเครื่อง
แคช	LocMemCache
ไฟล์นิ่ง	Tailwind CSS, Chart.js, JavaScript ของเทมเพลต
แพลตฟอร์ม	Render (แผนฟรี) เข้าถึงได้ที่ https://yumpakjure.onrender.com/
ที่เก็บซอร์สโค้ด	https://github.com/tlemtv098-dotcom/YumparkjureQRsong
ระบบควบคุมเวอร์ชัน	Git (270 คอมมิต)

โครงสร้างโปรเจกต์แยกตามหน้าที่ของ Django ไฟล์ yum_jukebox/settings.py เก็บค่าตั้งและรวม URL แอป music ประกอบด้วยโมเดล ฟอร์ม วิว เทมเพลต และคำสั่งจัดการฐานข้อมูล รวมถึงชุดทดสอบที่แยกตามหัวข้อ จำนวน 9 ไฟล์

3.2 โมเดลและโครงสร้างข้อมูล

โมเดลทั้ง 9 โมเดลอยู่ในไฟล์ music/models.py รายละเอียดของแต่ละโมเดลดังตาราง

โมเดล	ฟิลด์หลัก	ความสัมพันธ์
Profile	user, role, avatar, phone, created_at, updated_at	One-to-One -> User (สร้างอัตโนมัติด้วยสัญญาณ post_save)
Genre	name, slug, description, created_at	ปลายทางของ Many-to-Many จาก Playlist และ SongQueue
Tag	name, slug, created_at	ปลายทางของ Many-to-Many จาก Playlist และ SongQueue
Playlist	user, name, description, songs (JSON), is_public, created_at, updated_at	One-to-Many (user -> User), Many-to-Many -> Genre/Tag

โมเดล	ฟิลด์หลัก	ความสัมพันธ์
PlaylistSong	playlist, song, position, added_at	ตารางเชื่อม Playlist <-> SongQueue แบบ Many-to-Many ผ่านตารางกลาง
SongQueue	title, video_id, thumbnail, channel, audio_url, artwork, requested_by, client_id, is_played, created_at	One-to-Many (ผู้ถูกบันทึกคำขอ), Many-to-Many -> Genre/Tag
BlockedVideo	video_id, reason, created_at	ตัวกรอง video_id ที่ฝังไม่ได้
GoodVideo	video_id, plays, title, updated_at	บันทึกวิดีโอที่เล่นได้จริง
ClientLog	session, ua, event, detail, created_at	บันทึกเหตุการณ์ฝั่งผู้ใช้

การเปลี่ยนแปลงโครงสร้างข้อมูลทั้งหมดถูกบันทึกเป็น migration จำนวน 10 ไฟล์ ตั้งแต่ 0001_initial จนถึง 0010_playlistsong เมื่อนำขึ้นใช้งานระบบจะรันคำสั่ง migrate ก่อนเริ่ม Gunicorn ตามที่กำหนดไว้ใน Procfile

3.3 ระบบสมาชิกและการยืนยันตัวตน

การสมัครสมาชิกใช้ RegisterForm ที่สืบทอดจาก UserCreationForm ของ Django เพิ่มฟิลด์อีเมล ชื่อ นามสกุล เบอร์โทร และคำยืนยันรหัสผ่าน เมื่อบันทึกสำเร็จ สัญญาณ post_save จะสร้างแถว Profile ให้อัตโนมัติ และวิวจะกำหนดบทบาทเป็น customer จากนั้นจึงเรียก login เพื่อเข้าสู่ระบบทันที

การเข้าสู่ระบบใช้ LoginView ของ Django ร่วมกับ ThaiLoginForm ที่ปรับข้อความแจ้งเตือนเป็นภาษาไทย เส้นทาง /accounts/logout/ ใช้ LogoutView และพากลับไปหน้าเข้าสู่ระบบเมื่อออกจากระบบ

3.4 การควบคุมสิทธิ์ด้วย role_required

```
def role_required(allowed_roles):
    def decorator(view_func):
        @login_required
        def _wrapped_view(request, *args, **kwargs):
            if not hasattr(request.user, 'profile'):
                return redirect('player')
            if request.user.profile.role not in allowed_roles:
                return redirect('player')
            return view_func(request, *args, **kwargs)
        return _wrapped_view
    return decorator
```

เดคคอเรเตอร์นี้ถูกใช้กับวิวจัดการผู้ใช้ แนวเพลง แท็ก แดชบอร์ด และการส่งออกข้อมูล
ทำให้ผู้ที่ไม่มีสิทธิ์เข้าถึงไม่ได้แม้จะพิมพ์ URL โดยตรง และไม่ได้พึ่งการซ่อนเมนูเพียงอย่างเดียว

3.5 การทำ CRUD

ระบบทำ CRUD ได้ 5 กลุ่ม ได้แก่ ผู้ใช้ แนวเพลง แท็ก เพลย์ลิสต์ และคิวเพลง ทุกกลุ่มใช้ฟอร์มของ Django ร่วมกับเดคคอเรเตอร์ role_required ทุกการแก้ไขมีการตรวจสอบความถูกต้องก่อนบันทึก และแสดงข้อความแจ้งเตือน ผ่านระบบ messages ของ Django เมื่อบันทึกสำเร็จหรือไม่สำเร็จ

- ผู้ใช้: เพิ่มผู้ใช้ แก้ไขข้อมูลและบทบาท ปิดใช้งานชั่วคราว และลบผู้ใช้
- แนวเพลง: เพิ่ม แก้ไข ลบ พร้อมช่องค้นหาและการแบ่งหน้า
- แท็ก: เพิ่ม แก้ไข ลบ พร้อมช่องค้นหาและการแบ่งหน้า
- เพลย์ลิสต์: สร้าง เพิ่มเพลง ลบเพลง และสลับลำดับเพลงในเพลย์ลิสต์
- คิวเพลง: เพิ่มหน้าเดิมหรือต่อท้าย ย้ายลำดับ ทำเครื่องหมายว่าเล่นแล้ว และล้างคิว

3.6 การค้นหาเพลงผ่าน YouTube Data API v3

ฟังก์ชัน youtube_api_search เรียก https://www.googleapis.com/youtube/v3/search โดยส่งพารามิเตอร์ part=snippet, type=video, regionCode=TH, videoCategoryId=10 (หมวดเพลง), videoEmbeddable=true และ videoSyndicated=true การกำหนด videoEmbeddable ทำให้ได้เฉพาะวิดีโอที่ฝังในหน้าเว็บได้จริง ซึ่งเป็นการกรองปัญหาที่พบบ่อยที่สุดของระบบจูเกะซ์

ระบบรองรับคีย์หลายตัวโดยอ่านจากตัวแปรสภาพแวดล้อม YOUTUBE_API_KEYS คั่นด้วยเครื่องหมายจุลภาค เมื่อคีย์ตัวหนึ่งโควตาหมดหรือถูกจำกัดอัตราการเรียก ระบบจะลองคีย์ถัดไปอัตโนมัติ และกำหนดเวลารอของคำขอไว้ที่ 8 วินาที เมื่อไม่มีคีย์หรือเรียกไม่สำเร็จ ระบบจะใช้รายการเพลงสำรองที่เตรียมไว้ในโค้ดแทน เพื่อให้หน้าค้นหายังใช้งานได้ต่อ

หลังได้ผลลัพธ์ ระบบกรองซ้ำอีกชั้นฝั่งเซิร์ฟเวอร์ ได้แก่ ตรวจรูปแบบ video_id ด้วยนิพจน์ปกติ ข้ามวิดีโอที่อยู่ในตาราง BlockedVideo ข้ามชื่อที่ดูเป็นเนื้อหาสร้างด้วย AI และข้ามคลิปที่ไม่ใช่เพลง การกรองชั้นนี้ทำให้ผลลัพธ์ที่แสดงแก่ลูกค้าสะอาดกว่าการกรองฝั่งหน้าเว็บ

3.7 การอัปโหลดไฟล์และรูปภาพ

ระบบรองรับการอัปโหลดไฟล์ 2 จุด ได้แก่ รูปประจำตัวของผู้ใช้ เก็บในฟิลด์ Profile.avatar และภาพปกของเพลง เก็บในฟิลด์ SongQueue.artwork โดยฟังก์ชัน clean_avatar ตรวจสอบ 3 เงื่อนไขคือ ขนาดไฟล์ไม่เกิน 2 MB ชนิดไฟล์ต้องเป็น JPEG, PNG หรือ WebP และความกว้างสูงไม่เกิน 1000 พิกเซล การตรวจขนาดภาพใช้ Pillow ซึ่งเป็นแพ็กเกจที่ติดตั้งไว้ใน requirements.txt

3.8 ระบบแบ่งหน้า (Pagination)

หน้าจัดการผู้ใช้ แนวเพลง และแท็ก ใช้ Paginator ของ Django หน้าละ 20 รายการ รับค่าหน้าจากพารามิเตอร์ ?page= และใช้เมธอด get_page เพื่อให้เลขหน้าที่เกินจำนวนหน้าจริงถูกจัดการอย่างปลอดภัย หน้าจัดการผู้ใช้ยังรองรับการค้นหาจากชื่อผู้ใช้และอีเมล พร้อมตัวกรองตามบทบาทและสถานะการใช้งาน

3.9 การตรวจสอบความถูกต้องของฟอร์ม

การตรวจสอบทำทั้งจากตัวตรวจสอบมาตรฐานของ Django และจากเมธอด clean เพิ่มเติม ดังนี้

- อีเมลต้องไม่ซ้ำกับบัญชีที่มีอยู่ (clean_email)

- เบอร์โทรต้องเป็นตัวเลขเท่านั้น โดยตัดเครื่องหมายขีดและช่องว่างออกก่อนตรวจ (clean_phone)
- ความยาวชื่อแนวเพลงไม่เกิน 50 ตัวอักษร และ slug ต้องไม่ซ้ำ
- ไฟล์รูปภาพต้องไม่เกิน 2 MB ต้องเป็น JPEG, PNG หรือ WebP และความกว้างสูงไม่เกิน 1000 พิกเซล (clean_avatar)
- รหัสผ่านต้องผ่านตัวตรวจสอบความซับซ้อนของ Django และตรงกับคำยืนยัน

3.10 REST API

ระบบเปิด REST API จำนวน 26 เส้นทางภายใต้ /api/ ทุกเส้นทางตอบกลับเป็น JSON ยกเว้นสองเส้นทางที่คืนไฟล์ คือ /api/audio/ ที่คืนสตรีมเสียง และ /qr.png ที่คืนภาพรหัส QR การทำงานร่วมกันระหว่างหน้าเว็บกับ API เกิดจากการเรียกแบบไม่ต้องโหลดหน้าใหม่ทั้งหน้า (AJAX) ผ่าน fetch ในไฟล์ JavaScript ของเทมเพลต

กลุ่ม	ตัวอย่าง endpoint	หน้าที่
ค้นหา	/api/search/ , /api/suggest/ , /api/hits/	ค้นหาเพลง เติมคำแนะนำ และดึงเพลงยอดนิยม
จัดการคิว	/api/add/ , /api/add-front/ , /api/queue/ , /api/queue/move/	เพิ่มเพลงต่อคิวหรือต่อหน้า ดูคิว และสลับลำดับ
สถานะเพลง	/api/played// , /api/clear/ , /api/duration/	ทำเครื่องหมายว่าเล่นแล้ว ล้างคิว และอ่านความยาวเพลง
เพลงของฉัน	/api/my-songs/ , /api/my-songs//delete/	ดูและลบเพลงที่ตนเป็นผู้ขอ
เพลย์ลิสต์	/api/playlists/ , /api/playlists//load/ , /api/playlists//add-song/	สร้าง โหลด และเติมเพลงลงเพลย์ลิสต์
ดูแลระบบ	/api/block// , /api/unblock// , /api/stats/	บล็อกหรือปลดบล็อกวิดีโอ และดูสถิติรวม
บันทึกเหตุการณ์	/api/clientlog/ , /api/clientlog/recent/	บันทึกและอ่านเหตุการณ์ฝั่งผู้ใช้
สื่อและอื่น ๆ	/api/dashboard/stats/ , /api/audio/ , /api/ai/recommend/	สถิติแดชบอร์ด สตรีมเสียง และคำแนะนำเพลง

3.11 แดชบอร์ดและกราฟ

หน้า /dashboard/ แสดงการ์ดสรุป 4 ใบ ได้แก่ จำนวนผู้ใช้ทั้งหมด ผู้ใช้ที่ใช้งานอยู่ จำนวนแนวเพลง และจำนวนเพลงในคิว จากนั้นเป็นกราฟ 4 กราฟที่สร้างด้วย Chart.js ได้แก่ กราฟแท่งแนวนอนแสดงจำนวนเพลงแยกตามแนวเพลง กราฟเส้นแสดงจำนวนคำขอเพลงย้อนหลัง 7 วัน กราฟแท่งแนวนอนแสดงเพลงยอดนิยม 10 อันดับแรก และกราฟวงแหวนแสดงสัดส่วนสถานะคิวเพลง ข้อมูลสำหรับกราฟถูกคำนวณและแปลงเป็น JSON ในวิวฝั่งเซิร์ฟเวอร์ ไม่ใช่คำนวณใน JavaScript ของหน้าเว็บ ทำให้ตรวจสอบความถูกต้องได้จากชุดทดสอบ

3.12 การส่งออกข้อมูล

หน้าแดชบอร์ดมีปุ่มส่งออกข้อมูล 2 ปุ่มที่ใช้งานได้จริง ได้แก่ /export/playlists/csv/ ที่ส่งออกรายการเพลย์ลิสต์ และ /export/queue/csv/ ที่ส่งออกคิวเพลง ทั้งสองเส้นทางอยู่ภายใต้เดคคอเรเตอร์ role_required และคืนไฟล์ชนิด text/csv ให้ดาวน์โหลดทันที ส่วนปุ่มส่งออกแดชบอร์ดเป็น PDF เชื่อมต่อแล้ว โดยเรียก window.print() ให้ผู้ใช้บันทึกเป็น PDF จากเบราว์เซอร์

3.13 ความปลอดภัยที่เพิ่มเติม

- ตรวจสอบสิทธิ์ฝั่งที่เซิร์ฟเวอร์ด้วย role_required ไม่พึ่งเพียงการซ่อนเมนู
- ตรวจขนาดและชนิดไฟล์ที่อัปโหลดก่อนบันทึกลงดิสก์
- ป้องกันการอ่านไฟล์นอกโฟลเดอร์ media ด้วยการตรวจพาธด้วย os.path.commonpath ใน yum_jukebox/urls.py
- ป้องกัน CSRF ด้วย CsrfViewMiddleware และมีชุดทดสอบ XSS แยก 10 เคส
- เตือนเมื่อ SECRET_KEY ยังเป็นค่าสำรองในโค้ด และเตือนเมื่อ DEBUG เปิดอยู่ในสภาพแวดล้อมการใช้งานจริง
- ใช้ WhiteNoise เสิร์ฟไฟล์นิ่งและ Gunicorn เป็นเว็บเซิร์ฟเวอร์ในการใช้งานจริง

3.14 การนำขึ้นใช้งานจริง

ระบบถูกนำขึ้นใช้งานบนแพลตฟอร์ม Render ที่ https://yumpakjure.onrender.com/
ซึ่งเข้าถึงได้จากอินเทอร์เน็ตทั่วโลก ไฟล์ Procfile กำหนดให้รันคำสั่ง migrate และ ensure_admin ก่อนเริ่ม Gunicorn ทุกครั้งที่เริ่มระบบ นอกจากนี้ยังมี Dockerfile และไฟล์ railway.toml
เตรียมไว้สำหรับการติดตั้งแบบคอนเทนเนอร์บนแพลตฟอร์มอื่น ทั้งนี้เว็บไซต์ที่ใช้งานจริงทำงานบน Render โดยตรง

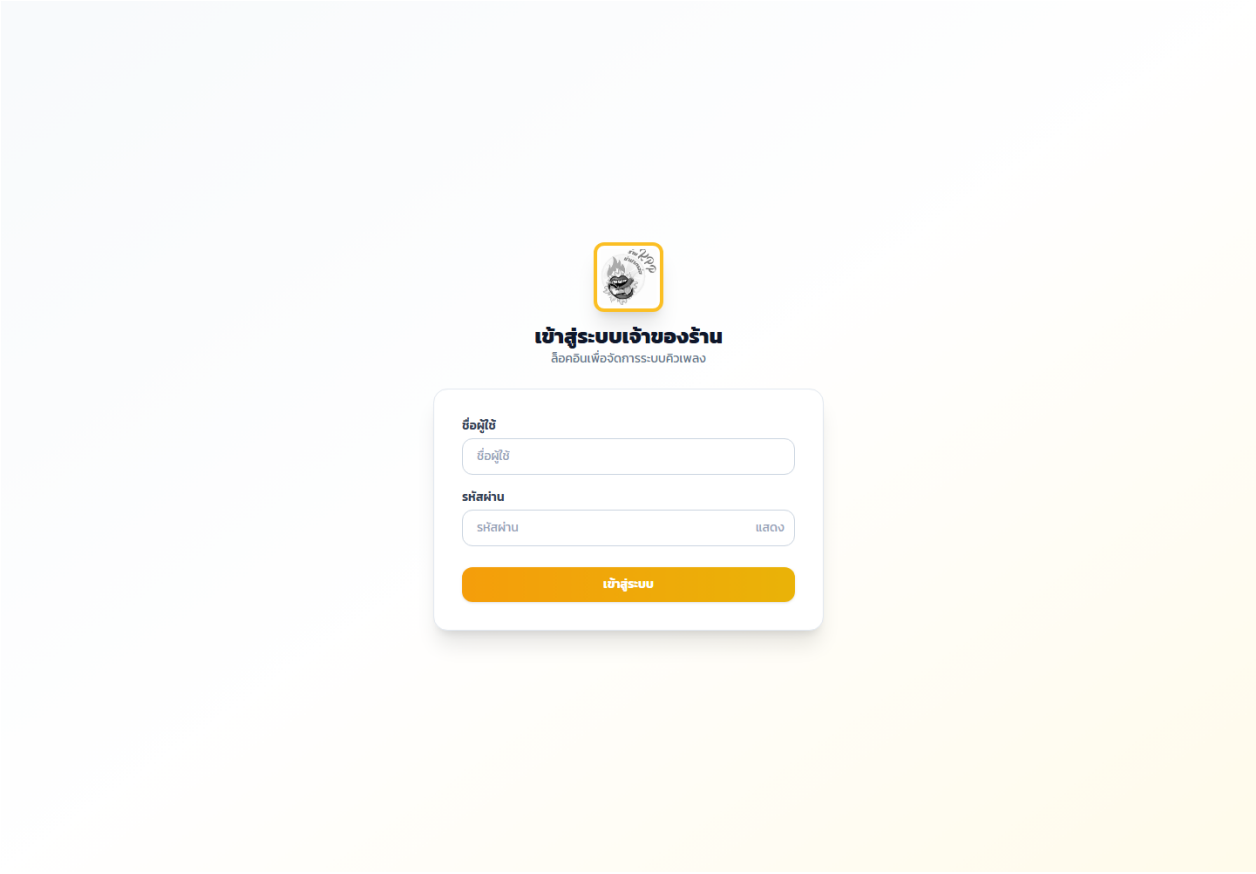
บทที่ 4 ผลการพัฒนาและภาพหน้าจอ

4.1 ความครบถ้วนตามข้อกำหนดของงาน

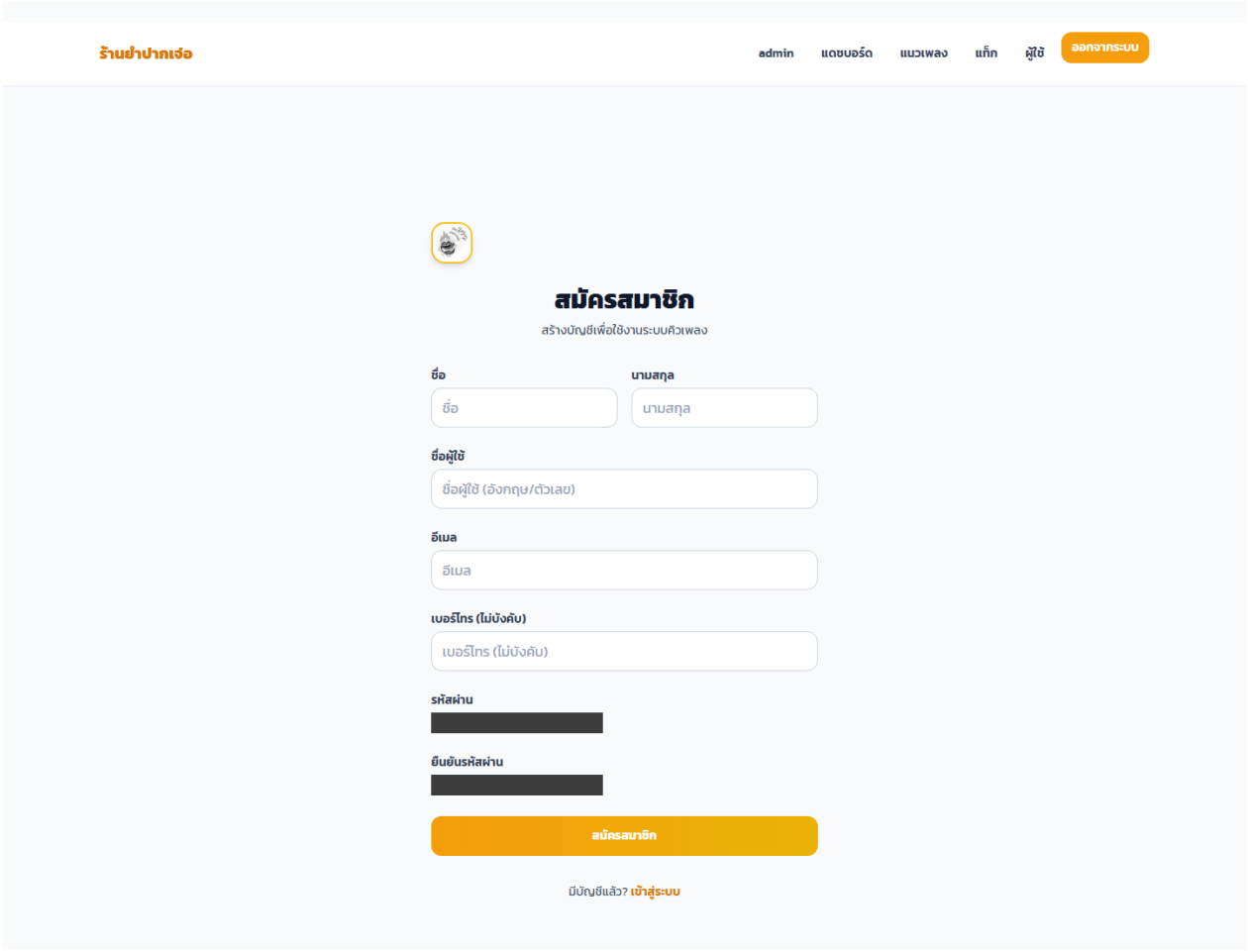
ข้อกำหนดของงาน	สิ่งที่ระบบทำได้	ตำแหน่งในโค้ด
สมัครสมาชิก / เข้าสู่ระบบ / ออกจากระบบ	ครบทั้งสามส่วน พร้อมฟอร์มภาษาไทยและข้อความแจ้งเตือน	music/views_auth.py , music/views.py (ThaiLoginForm)
อย่างน้อย 2 บทบาท พร้อมควบคุมสิทธิ์จริง	3 บทบาท (admin, staff, customer) คุมด้วย role_required	music/views_auth.py บรรทัด role_required
CRUD ด้วย Django Model และฐานข้อมูล	CRUD ครบ ทั้งผู้ใช้ แนวเพลง แท็ก เพลย์ลิสต์ และคิวเพลง	music/views_auth.py , music/views.py
ระบบค้นหา	ค้นหาเพลงผ่าน YouTube Data API v3 พร้อมตัวกรองฝั่งเซิร์ฟเวอร์	music/views.py ฟังก์ชัน youtube_api_search และ search_youtube
อัปโหลดไฟล์ / รูปภาพ	อัปโหลดรูปประจำตัว (avatar) และภาพปกเพลง (artwork) พร้อมตรวจขนาดและชนิดไฟล์	music/forms.py ฟังก์ชัน clean_avatar
แบ่งหน้า (Pagination)	Paginator หน้าละ 20 รายการ ทั้งผู้ใช้ แนวเพลง และแท็ก	music/views_auth.py ฟังก์ชัน user_list_view, genre_list_view, tag_list_view
ความสัมพันธ์ระหว่างโมเดล	มีครบ One-to-One, One-to-Many และ Many-to-Many	music/models.py
ตรวจสอบความถูกต้องของฟอร์ม	ตรวจอีเมลซ้ำ เบอร์โทรผิดรูป ขนาดไฟล์ ชนิดไฟล์ ความยาว slug	music/forms.py
ส่วนติดต่อผู้ใช้ที่ใช้งานได้จริง	หน้าเข้าสู่ระบบ สมัครสมาชิก แถบเมนู และหน้าติดต่อ/แจ้งปัญหาในหน้าเครื่องเล่น	music/templates/base.html, music/templates/music/player.html
ให้บริการบนอินเทอร์เน็ต	ใช้งานจริงที่ Render ผ่าน Gunicorn และ WhiteNoise	Procfile, Dockerfile, yum_jukebox/settings.py

4.2 ภาพหน้าจอของระบบ

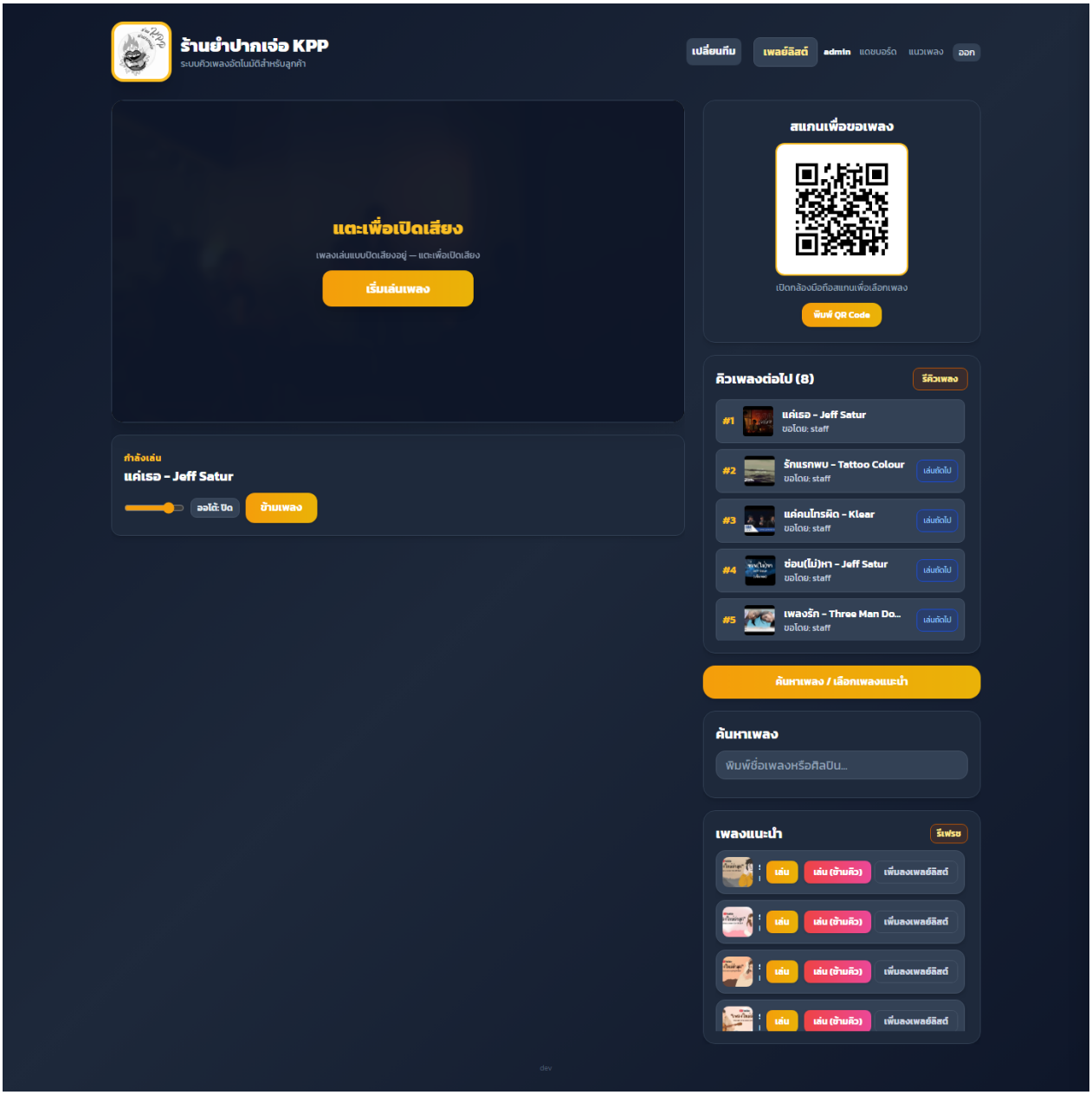
ภาพทั้งหมดต่อไปนี้ถูกบันทึกจากระบบที่รันจริงด้วยข้อมูลตัวอย่างที่สร้างจากคำสั่ง ensure_admin --with-demo โดยเข้าสู่ระบบในส่วนที่ต้องการสิทธิ์ด้วยบัญชี admin เพื่อให้เห็นหน้าจอจริงทั้งหมด



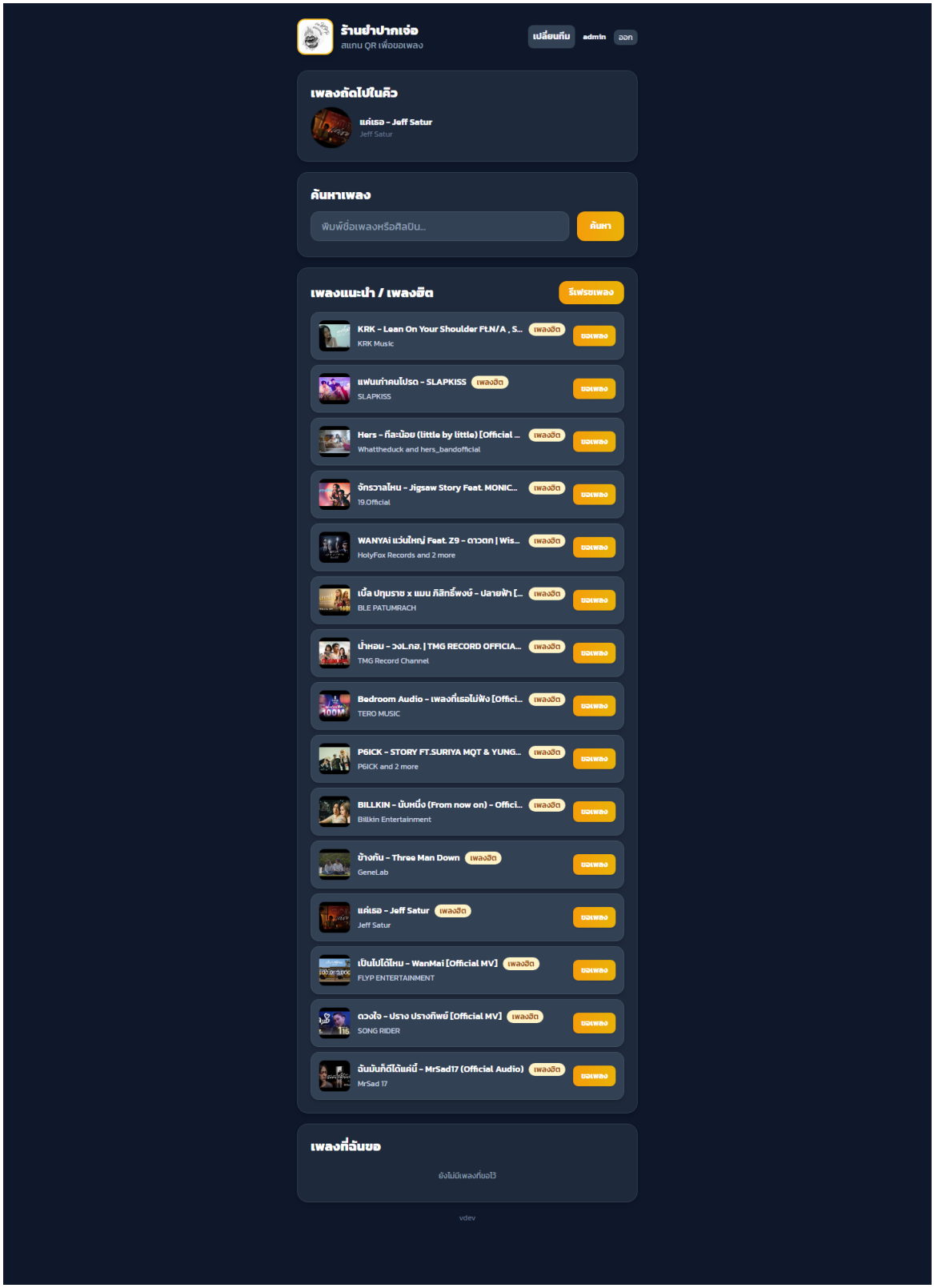
ภาพที่ 4-1 หน้าเข้าสู่ระบบ พร้อมข้อความแจ้งเตือนภาษาไทย



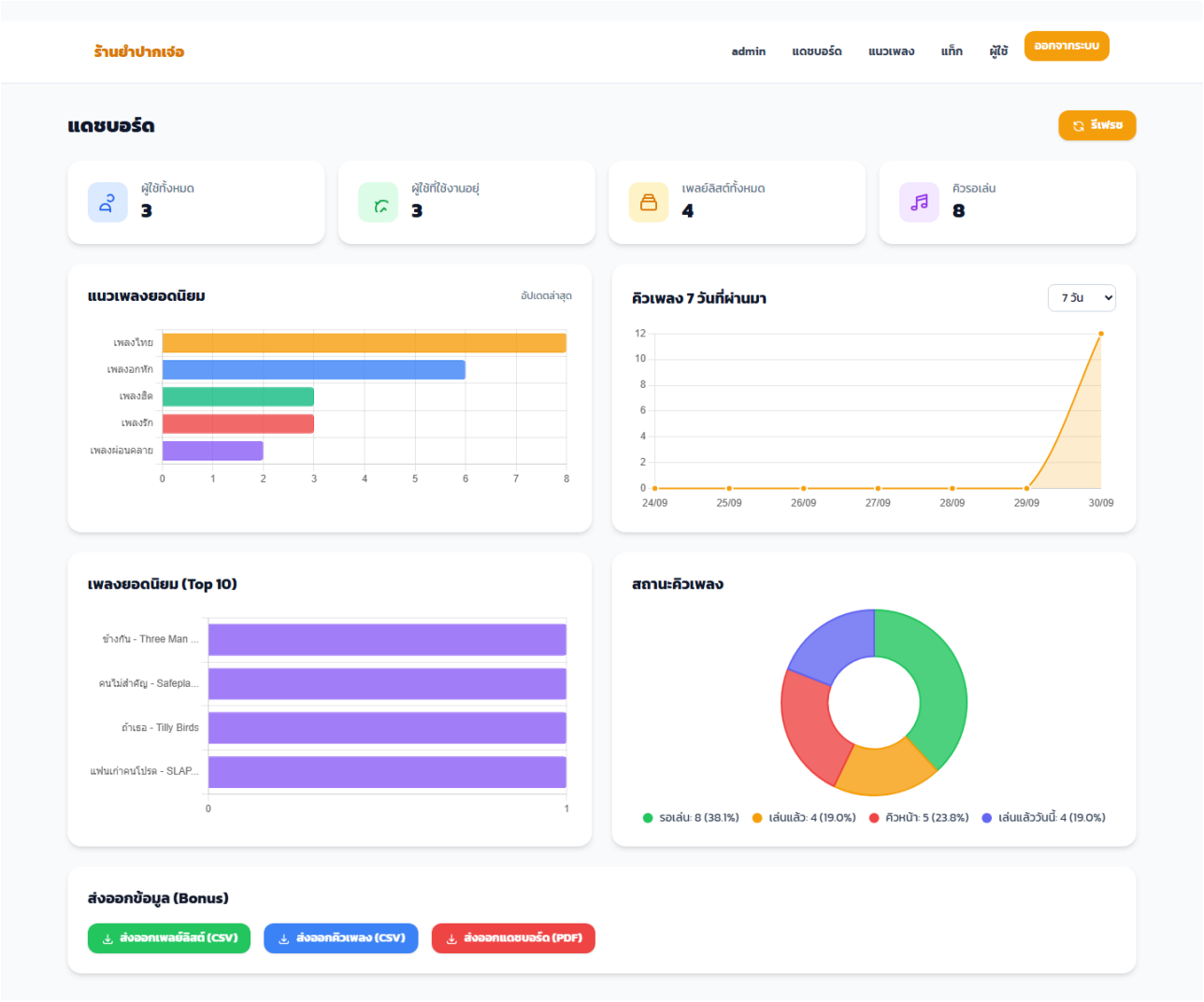
ภาพที่ 4-2 หน้าสมัครสมาชิก มีช่องอีเมล ชื่อ นามสกุล เบอร์โทร และคำยืนยันรหัสผ่าน



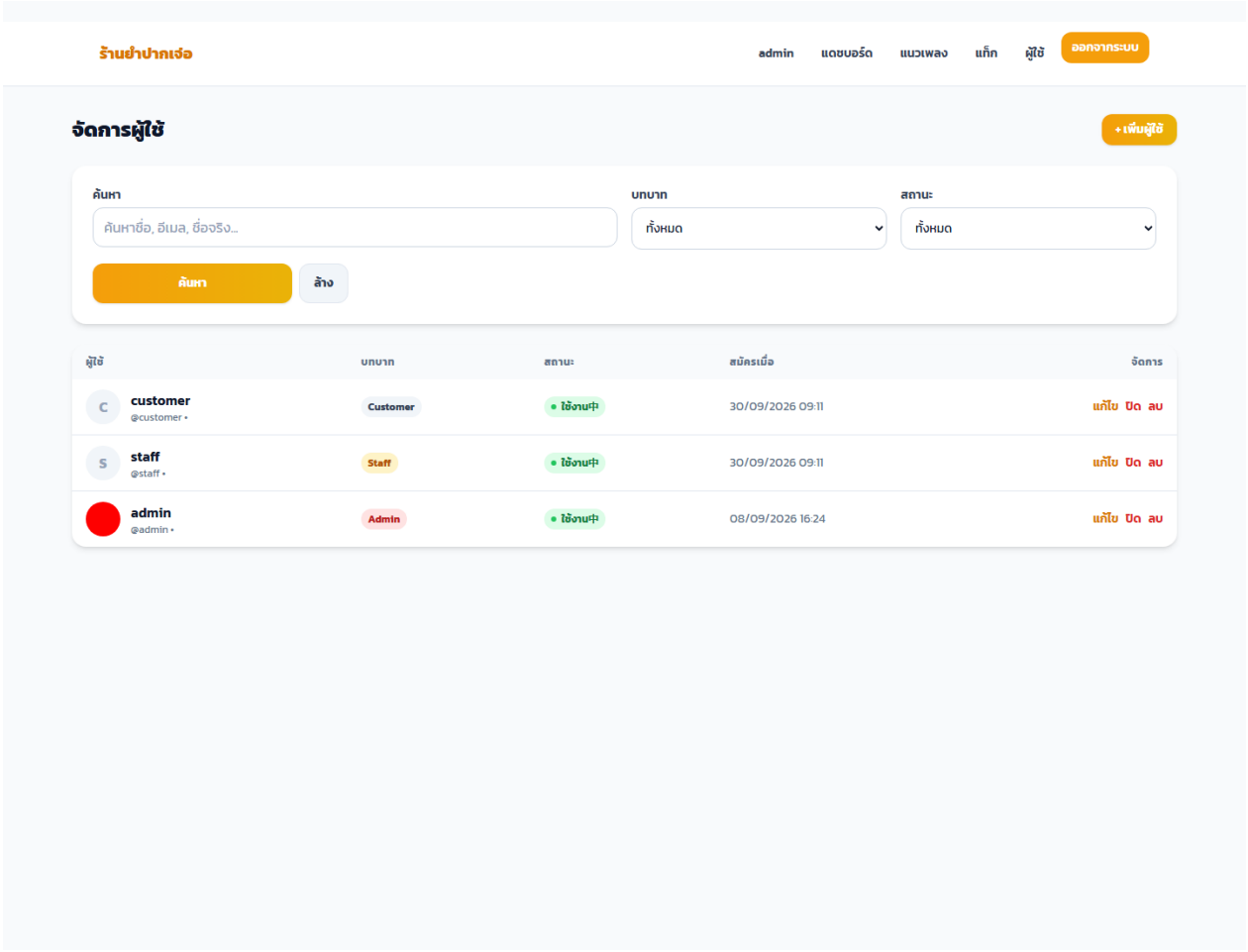
ภาพที่ 4-3 หน้าเครื่องเล่นจูเกะซ์ขณะเข้าสู่ระบบในฐานะ admin เมื่อกดเล่นแล้วระบบจะเล่นเพลงถัดไป



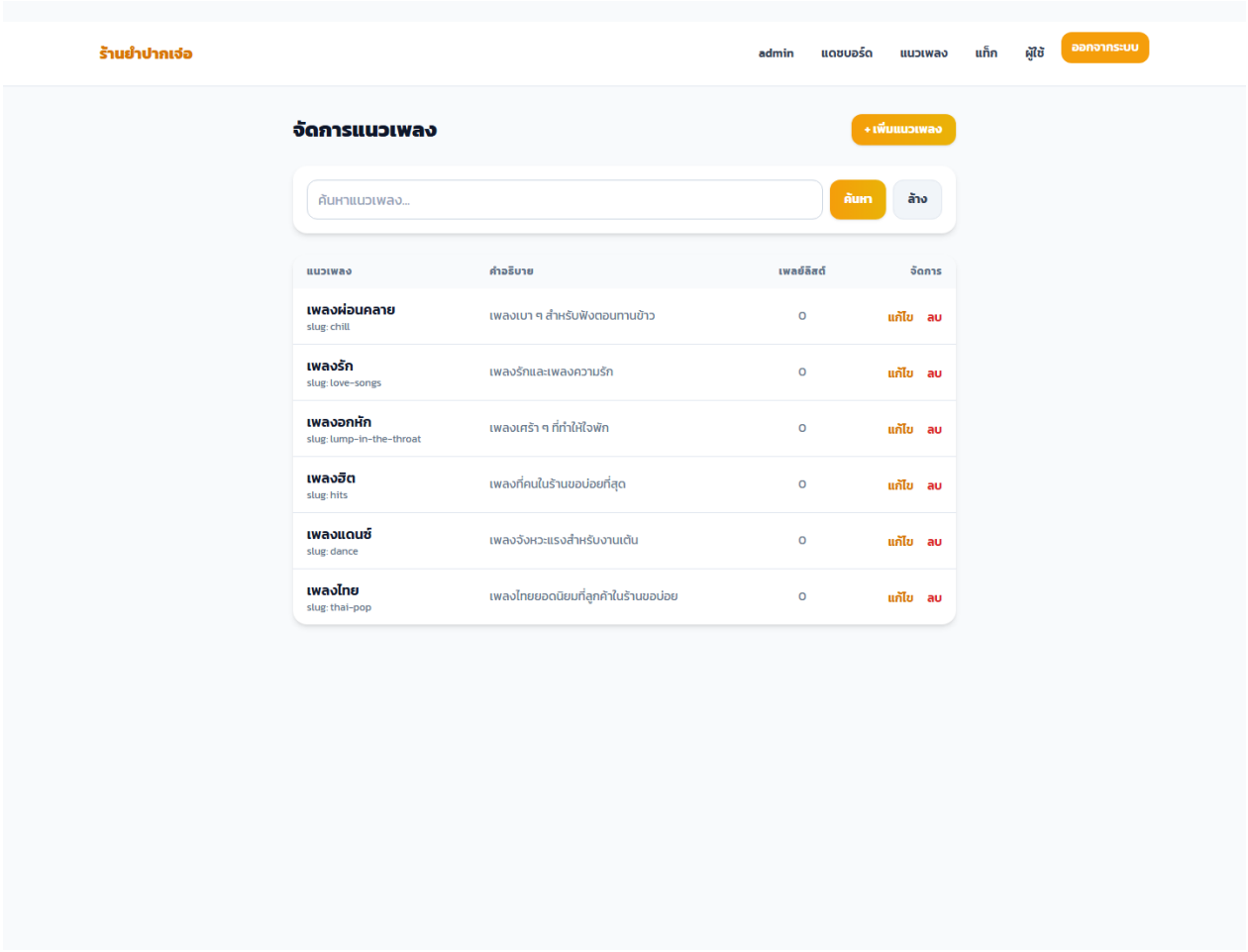
ภาพที่ 4-4 หน้าขอเพลง ผลลัพธ์จาก YouTube พร้อมรูปปก ช่องใส่แท็ก และปุ่มขอเพลง



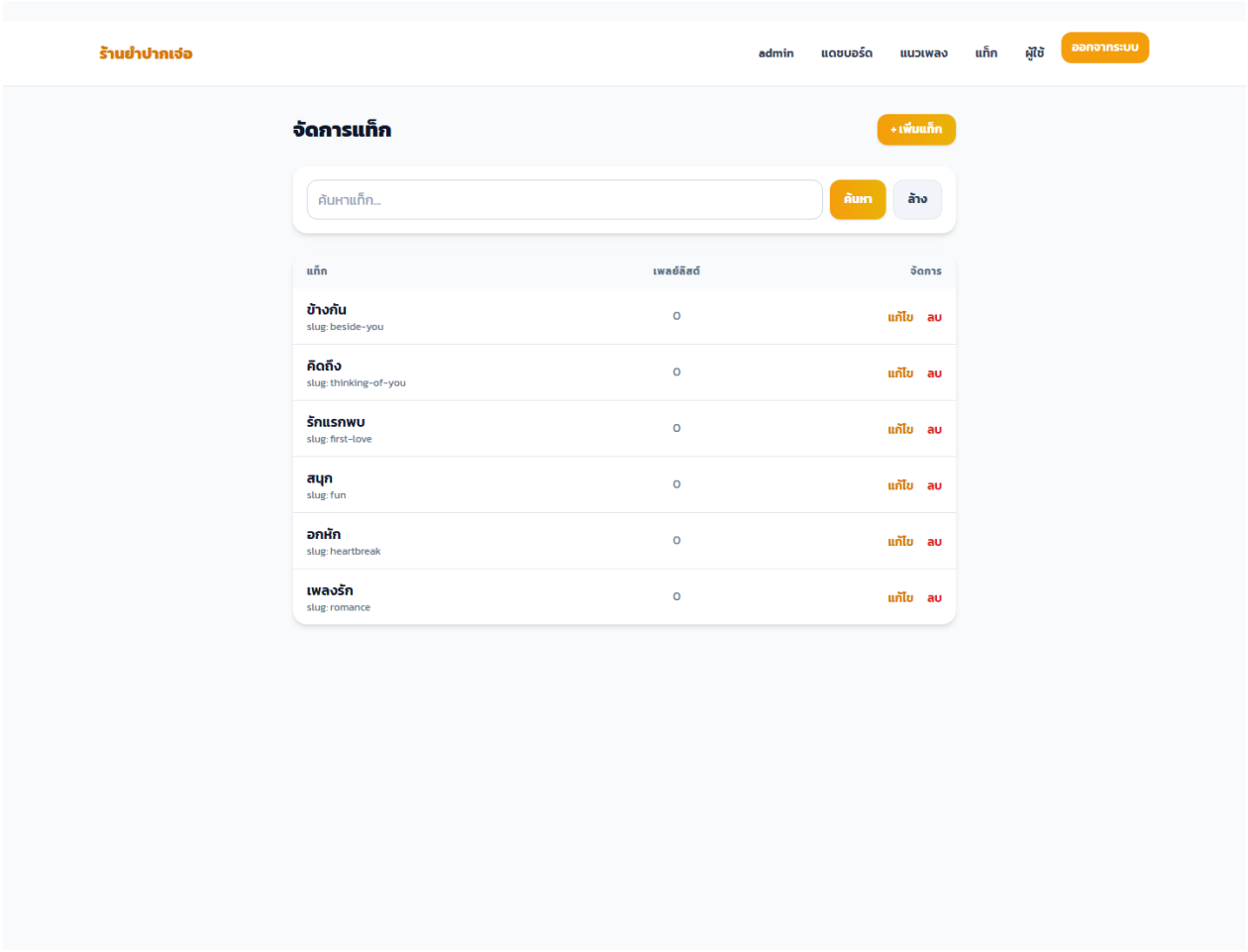
ภาพที่ 4-5 แดชบอร์ดของผู้ดูแลระบบ ประกอบด้วยการ์ดสรุป 4 ใบ กราฟ 4 กราฟ และปุ่มส่งออกข้อมูล



ภาพที่ 4-6 หน้าจัดการผู้ใช้ มีช่องค้นหา ตัวกรองตามบทบาทและสถานะ และปุ่มแก้ไข ปิดใช้งาน ลบ



ภาพที่ 4-7 หน้าจัดการแนวเพลง (CRUD) พร้อมช่องค้นหาและปุ่มเพิ่มแนวเพลง



ภาพที 4-8 หน้าจัดการแท็ก (CRUD) ซึ่งใช้รูปแบบเดียวกับหน้าแนวเพลง

บทที่ 5 การทดสอบ

5.1 การทดสอบอัตโนมัติ

การทดสอบอัตโนมัติเขียนด้วย unittest ของมาตรฐาน Django จำนวน 386 เคส แยกเป็น 25 โมดูลตามหัวข้อ เพื่อให้เห็นชัดว่าแต่ละส่วนถูกตรวจอะไรบ้าง คำสั่งที่ใช้รันคือ

```
venv\Scripts\python.exe manage.py test
```

ผลการรันครั้งล่าสุด

```
Found 386 test(s).
Ran 386 tests in 406.5s

OK
```

โมดูลการทดสอบ	หน้าทีทีตรวจ	จำนวนเคส
tests.py	เส้นทางหลัก การขอเพลง คิวเพลง การค้นหา เพลย์ลิสต์ API	156
tests_demo_accounts.py	คำสั่ง ensure_admin และบัญชีทดสอบ 3 บทบาท	28
tests_production_config.py	ค่าตั้งค่าสำหรับ production (SECRET_KEY, DEBUG, security)	15
tests_nav_customer.py	เมนูและสิทธิ์ของบทบาท customer	11
tests_xss.py	การป้องกัน XSS ในช่องค้นหาและข้อมูลที่แสดงผล	10
tests_dashboard.py	แดชบอร์ดและการคำนวณค่าสถิติ	8
tests_media.py	การอัปโหลดไฟล์และการเสิร์ฟไฟล์ media	7
tests_nav.py	เมนูของบทบาท staff และ admin	6
tests_profile_signal.py	สัญญาณ post_save สร้าง Profile อัตโนมัติ	6
รวม	ผลการรัน: Ran 386 tests — OK	386

5.2 รายละเอียดของการทดสอบในแต่ละด้าน

- tests.py จำนวน 157 เคส ครอบคลุมเส้นทางหลักทั้งหมด ทั้งหน้าเว็บ การขอเพลง การจัดการคิว การค้นหาและเพลย์ลิสต์ รวมถึงกรณีผิดปกติ เช่น ส่ง JSON ไม่ถูกต้อง หรือไม่มีสิทธิ์เข้าถึง
- tests_demo_accounts.py จำนวน 28 เคส ตรวจว่าคำสั่ง ensure_admin สร้างบัญชีทั้งสามบทบาทและข้อมูลตัวอย่างได้ถูกต้อง และรันซ้ำแล้วไม่ทำให้ข้อมูลเพียน
- tests_production_config.py จำนวน 15 เคส ตรวจการตั้งค่าที่จำเป็นต่อการใช้งานจริง เช่น การเตือนเมื่อ SECRET_KEY เป็นค่าสำรอง และการปิด DEBUG ในสภาพแวดล้อมการใช้งานจริง

- tests_nav_customer.py และ tests_nav.py จำนวน 17 เคส ตรวจว่าเมนูและการเข้าถึงหน้าต่าง ๆ เป็นไปตามบทบาทจริง
- tests_xss.py จำนวน 10 เคส ตรวจการป้องกันสคริปต์ที่ส่งมากับคำค้นและข้อมูลที่แสดงผล
- tests_media.py จำนวน 7 เคส ตรวจการอัปโหลดไฟล์และการเสิร์ฟไฟล์ในโฟลเดอร์ media
- tests_dashboard.py จำนวน 8 เคส ตรวจการคำนวณสถิติและการควบคุมสิทธิ์ของแดชบอร์ด
- tests_profile_signal.py จำนวน 6 เคส ตรวจว่าสัญญาณ post_save สร้าง Profile ให้ผู้ใช้ใหม่เสมอ

5.3 ผลการทดสอบด้วยมือ

นอกจากการทดสอบอัตโนมัติแล้ว ยังทำการทดสอบด้วยมือตามตารางต่อไปนี้ เพื่อยืนยันพฤติกรรมที่ผู้ใช้เห็นจริง โดยเฉพาะกรณีที่การทดสอบอัตโนมัติตรวจไม่ได้ เช่น การเล่นเพลงบนอุปกรณ์จริงและการแสดงผลบนหน้าจอ

ลำดับ	สิ่งที่ต้องทำ	ผลที่คาดหวัง
1	เปิดหน้าเข้าสู่ระบบแล้วกรอก admin / 12345678	เข้าสู่ระบบได้ เมนูด้านบนเพิ่มเมนูจัดการ
2	กดออกจากระบบ แล้วกรอกรหัสผิด	ขึ้นข้อความชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง
3	สมัครสมาชิกใหม่ด้วยอีเมลที่ยังไม่เคยใช้	สมัครได้ และเข้าสู่ระบบอัตโนมัติ
4	สมัครใหม่อีกครั้งด้วยอีเมลเดิม	ขึ้นข้อความ อีเมลนี้ถูกใช้งานแล้ว
5	กรอกเบอร์โทรศัพท์เป็นตัวอักษร	ขึ้นข้อความ เบอร์โทรต้องเป็นตัวเลขเท่านั้น
6	หน้า /request/ พิมพ์คำค้น แล้วกดค้นหา	แสดงผลลัพธ์เป็นรายการพร้อมรูปปกและช่องใส่แท็ก
7	กด ขอเพลง ที่ผลลัพธ์ใดผลลัพธ์หนึ่ง	เพลงถูกเพิ่มเข้าคิว และถูกลบออกจากรายการผลลัพธ์
8	กด ขอเพลง ซ้ำกับเพลงเดิม	ไม่เพิ่มซ้ำ ระบบตรวจจับเพลงที่อยู่ในคิวแล้ว
9	เปิดหน้า / แล้วกดเล่นเพลงถัดไป	เพลงถัดไปขึ้นเป็นเพลงที่กำลังเล่น และสถานะเปลี่ยนเป็นเล่นแล้ว
10	กดแท็กหนึ่งในรายการเพลงที่เล่นได้ล่าสุด	เพลงนั้นถูกโหลดเข้าคิวและเริ่มเล่น
11	กด ล้างคิวเพลง	คิวว่าง และปุ่มยืนยันถูกซ่อน
12	หน้า /dashboard/ ตรวจกราฟทั้งสี่	กราฟแท่งแนวนอน กราฟเส้น กราฟแท่ง Top 10 และกราฟวงแหวน แสดงข้อมูลตรงกับฐานข้อมูล
13	หน้า /management/users/ ค้นหาด้วยคำว่า admin	รายการเหลือเฉพาะผู้ใช้ที่ตรงกับคำค้น
14	หน้า /management/users/ เปลี่ยนบทบาทของผู้ใช้	บทบาทในตารางเปลี่ยนตามที่เลือก
15	หน้า /management/genres/ เพิ่มแนวเพลงใหม่	แนวเพลงใหม่ปรากฏในตารางพร้อม slug
16	ลบแนวเพลงที่ยังไม่ถูกใช้งาน	แนวเพลงถูกลบออกจากตาราง
17	หน้า /accounts/profile/ อัปโหลดรูปเกิน 2 MB	ขึ้นข้อความ ไฟล์รูปภาพต้องไม่เกิน 2 MB
18	อัปโหลดไฟล์ .txt เป็นรูปประจำตัว	ขึ้นข้อความ รองรับเฉพาะไฟล์ JPEG, PNG, WebP เท่านั้น

ลำดับ	สิงที่ต้องทำ	ผลที่คาดหวัง
19	เข้าสู่ระบบด้วย customer แล้วเปิด /management/users/	ถูกพากลับไปหน้าเครื่องเล่น พร้อมข้อความแจ้งเตือน
20	เปิด /qr.png	ได้ภาพรหัส QR ที่สแกนแล้วเปิดหน้าเว็บไซต์ได้

บทที่ 6 สรุปผล

6.1 สิ่งที่บรรลุตามวัตถุประสงค์

โครงการนี้พัฒนาระบบจูเกะซ์สำหรับร้านกาแฟที่ลูกค้าใช้สแกนคิวอาร์โค้ดเพื่อขอเพลงได้ด้วยตนเอง โดยไม่ต้องเดินไปที่เคาน์เตอร์ ระบบทำงานได้ครบตามข้อกำหนดของงานทุกข้อ คือ การสมัครสมาชิก เข้าสู่ระบบและออกจากระบบ การควบคุมสิทธิ์ตาม 3 บทบาท CRUD ด้วย Django Model และฐานข้อมูล ระบบค้นหา การอัปโหลดไฟล์ การแบ่งหน้า ความสัมพันธ์ระหว่างโมเดล การตรวจสอบความถูกต้องของฟอร์ม ส่วนติดต่อผู้ใช้ที่อ่านง่ายและใช้งานได้จริง และการให้บริการผ่านอินเทอร์เน็ต

นอกจากข้อกำหนดหลัก ระบบยังมีองค์ประกอบตามเกณฑ์พิจารณาพิเศษ ได้แก่

เกณฑ์พิจารณาพิเศษ	สถานะ	รายละเอียดที่มีในระบบ
REST API	มี	เส้นทางใต้ /api/ จำนวน 26 เส้นทาง ครอบคลุมการค้นหา คิวเพลง เพลย์ลิสต์ และสถิติ
การเชื่อมต่อ API ภายนอก	มี	YouTube Data API v3 สำหรับค้นหา และ YouTube oEmbed สำหรับข้อมูลวิดีโอ
แดชบอร์ดหรือกราฟ	มี	แดชบอร์ดพร้อมกราฟ 4 กราฟ วาดด้วย Chart.js
การค้นหาขั้นสูง	มี	กรองตามประเทศและหมวดเพลง ตรวจสถานะการฝังได้ กรองคีย์เวิร์ด และมีระบบเติมคำแนะนำอัตโนมัติ
การทดสอบอัตโนมัติ	มี	ชุดทดสอบ 386 เคส 25 โมดูล ผ่านทั้งหมด
การแจ้งเตือนทางอีเมล	ยังไม่มี	ระบบยังไม่ได้เชื่อมต่อบริการส่งอีเมล ระบุไว้ในแนวทางพัฒนาต่อ
AJAX	มี	หน้าเว็บเรียก API ด้วย fetch ไม่ต้องโหลดหน้าใหม่ทั้งหน้า
การส่งออกข้อมูล	มี	ส่งออกเพลย์ลิสต์และคิวเพลงเป็น CSV ได้จริง และปุ่มส่งออกแดชบอร์ดเป็น PDF เชื่อมต่อแล้วผ่าน window.print()
Docker	มีไฟล์	Dockerfile และ railway.toml เตรียมไว้ แต่เว็บไซต์จริงทำงานบน Render
CI/CD	ยังไม่มี	ยังไม่ได้ตั้งค่า workflow อัตโนมัติใน GitHub Actions
ความปลอดภัยเพิ่มเติม	มี	ตรวจสิทธิ์ฝั่งเซิร์ฟเวอร์ ตรวจไฟล์อัปโหลด ป้องกันการอ่านไฟล์นอกโฟลเดอร์ ป้องกัน XSS และมีชุดทดสอบความปลอดภัยแยก

ผลการทดสอบอัตโนมัติผ่านทั้งหมด 386 เคส ไม่มีเคสไหนล้มเหลว ระบบถูกนำขึ้นใช้งานจริงและเปิดให้ทดลองใช้งานได้จากอินเทอร์เน็ตที่ https://yumpakjure.onrender.com/

6.2 ข้อจำกัดที่ทราบของระบบ

ระบบยังมีข้อจำกัดที่ทราบอยู่หลายข้อ ผู้จัดทำรายงานไว้อย่างเปิดเผยทั้งที่สามารถตรวจสอบได้จากโค้ดจริง เพราะการนำเสนอเฉพาะสิ่งที่ดูดีอย่างเดียวจะทำให้ผู้ประเมินตรวจสอบแล้วไม่เชื่อถือระบบทั้งหมด

ข้อจำกัด	รายละเอียดและผลกระทบ	แนวทางแก้ไข
ฐานข้อมูลการใช้งานจริงย้ายไป PostgreSQL (Neon) แล้ว	เดิมระบบบน Render ใช้ SQLite บนดิสก์ชั่วคราว ข้อมูลจึงหายทุกครั้งที่ deploy ใหม่ ปัจจุบันตั้งค่า DATABASE_URL ชี้ไปยัง Neon Postgres แล้ว ข้อมูลจึงคงอยู่ข้ามการ deploy ส่วน SQLite (db.sqlite3) ยังใช้สำหรับการพัฒนาในเครื่องเท่านั้น	ไม่ต้องทำเพิ่มในส่วนนี้ — เหลือเพียงย้ายไฟล์อัปโหลดไปพื้นที่เก็บไฟล์ภายนอก
SECRET_KEY ของการใช้งานจริงตั้งเป็นตัวแปรสภาพแวดล้อมแล้ว	ในซอร์สโค้ดยังมีค่าสำรองไว้สำหรับพัฒนาในเครื่อง แต่บน Render ตั้ง SECRET_KEY เป็นสตริงสุ่มในตัวแปรสภาพแวดล้อมแล้ว โค้ดจะออกคำเตือนชัดเจนถ้าพบค่าสำรองในการใช้งานจริง	ไม่ต้องทำเพิ่มในส่วนนี้ — คงไว้เพียงคำเตือนในโค้ด
การเล่นเพลงบน iOS และ Safari มีข้อจำกัดที่ทราบ	YouTube IFrame API ของบางเครื่อง iOS ตอบกลับเป็น Error 153 ซึ่งเป็นนโยบายฝั่งผู้ใช้ของ YouTube ไม่ใช่ความผิดของเซิร์ฟเวอร์ ระบบเตรียมโหมด ?noapi=1 ไว้เป็นทางเลี่ยงให้เปลี่ยนไปใช้ iframe ธรรมดา และมีเส้นทางสำรองผ่าน Media Session	ใช้โหมด ?noapi=1 บนเครื่องที่มีปัญหา รายละเอียดอยู่ใน docs/ios-audio-notes.md
ไฟล์ที่อัปโหลดบน Render ไม่คงอยู่ถาวร	ไฟล์ที่อัปโหลดถูกเก็บไว้บนดิสก์ของ container และจะหายไปเมื่อมีการ deploy ใหม่ รูปประจำตัวและภาพปกเพลงที่อัปโหลดไว้จึงไม่อยู่ตลอดไป	ใช้พื้นที่เก็บไฟล์ภายนอก เช่น S3 หรือ Cloudinary แล้วเก็บแค่ URL ในฐานข้อมูล
ปุ่มส่งออกแดชบอร์ด (PDF) เชื่อมต่อแล้ว	ปุ่ม exportDashboardPdf บนแดชบอร์ดเรียก window.print() ให้ผู้ใช้เลือกบันทึกเป็น PDF ได้จากหน้าต่างพิมพ์ของเบราว์เซอร์ ส่วนการส่งออก CSV สองรายการใช้งานได้จริงที่ /export/playlists/csv/ และ /export/queue/csv/	ไม่ต้องทำเพิ่มในส่วนนี้

6.3 แนวทางพัฒนาต่อ

- ย้ายไฟล์ที่อัปโหลดไปยังพื้นที่เก็บไฟล์ภายนอก เช่น S3 หรือ Cloudinary เพื่อให้รูปภาพอยู่คงถาวรข้ามการ deploy (ส่วนฐานข้อมูลย้ายไป PostgreSQL แล้ว SECRET_KEY กับ DATABASE_URL ตั้งเป็นตัวแปรสภาพแวดล้อมแล้ว และปุ่มส่งออก PDF เชื่อมต่อแล้ว)
- เพิ่มการแจ้งเตือนทางอีเมลเมื่อคำขอเพลงเข้าคิว เพื่อให้ผู้ใช้ทราบลำดับคิวของตน
- เพิ่มการควบคุมโควตาของ YouTube Data API ให้เหมาะกับการใช้งานจริงที่มีผู้ใช้จำนวนมาก

ภาคผนวก

ก. ข้อมูลสำหรับเข้าใช้งานระบบ

รายการ	ค่า
เว็บไซต์ที่ใช้งานจริง	https://yumpakjure.onrender.com/
ที่เก็บซอร์สโค้ด	https://github.com/tlemtv098-dotcom/YumparkjureQRsong
จำนวนคอมมิตใน Git	242 คอมมิต
ชุดทดสอบอัตโนมัติ	386 เคส ผ่านทั้งหมด แยกเป็น 25 โมดูล
จำนวน URL route	54 route ใน music/urls.py (ไม่รวม /admin/ ของ Django)
จำนวน API endpoint	26 route อยู่ใต้ /api/
จำนวนโมเดล	9 โมเดลใน music/models.py
จำนวน migration	10 ไฟล์ ตั้งแต่ 0001_initial ถึง 0010_playlistsong
บทบาทผู้ใช้	admin, staff, customer
endpoint รหัส QR	/qr.png

รหัสคิวอาร์โค้ดสำหรับเปิดหน้าเว็บไซต์จากโทรศัพท์ เข้าถึงได้ที่ https://yumpakjure.onrender.com/qr.png

ภาพที่ ก-1 รหัสคิวอาร์โค้ดที่สร้างจากเส้นทาง /qr.png

ข. บัญชีทดสอบสำหรับผู้ประเมิน

ชื่อผู้ใช้	รหัสผ่าน	บทบาท	สิทธิ์ที่ใช้ได้
admin	12345678	admin (superuser)	ดูแล้วปเดอร์, แดชบอร์ด, จัดการผู้ใช้/แนวเพลง/แท็ก
staff	12345678	staff	จัดการคิวเพลงและเพลย์ลิสต์
customer	12345678	customer	ขอเพลง สร้างและจัดการเพลย์ลิสต์ของตน

ข้อควรทราบเรื่องรหัสผ่าน: คำสั่ง ensure_admin ที่ไม่มีตัวเลือกเพิ่มเติมจะตั้งรหัสผ่านของบัญชี admin เป็น 11111111 ส่วนเมื่อใช้ตัวเลือก --with-demo รหัสผ่านของบัญชีทั้งสามจะถูกตั้งเป็น 12345678 ตามค่าเริ่มต้นของตัวเลือกนี้ หากต้องการให้ตรงกับตารางข้างต้น ให้รันคำสั่งตามรูปแบบนี้

```
python manage.py ensure_admin
python manage.py ensure_admin --with-demo
```

ค. คู่มือการใช้งานย่อ

คู่มือการใช้งานฉบับเต็มจัดทำแยกเป็นอีกเล่มหนึ่งชื่อ คู่มือการใช้งาน.docx สรุปสาระสำคัญไว้ดังนี้

1. สมัครสมาชิก: เปิด https://yumpakjure.onrender.com/accounts/register/ กรอกอีเมล ชื่อ นามสกุล เบอร์โทรศัพท์ และรหัสผ่าน ระบบจะเข้าสู่ระบบให้อัตโนมัติ
2. เข้าสู่ระบบ: เปิดหน้าเข้าสู่ระบบ แล้วกรอกชื่อผู้ใช้และรหัสผ่าน
3. ขอเพลง: เปิดหน้า ขอเพลง พิมพ์ชื่อเพลงหรือชื่อศิลปิน แล้วกดค้นหา กดปุ่ม ขอเพลง ที่ผลลัพธ์ที่ต้องการ ระบบจะเพิ่มเพลงเข้าคิวของร้าน
4. ใช้หน้าเครื่องเล่น: เปิดหน้าแรกของเว็บไซต์ จะเห็นเครื่องเล่น เพลงที่กำลังเล่น คิวเพลงที่รอ และรายการเพลงที่เล่นล่าสุด กด เพลงถัดไป เพื่อเล่นเพลงถัดไปในคิว
5. เล่นเพลงที่เล่นได้ล่าสุด: กดแท็กในรายการด้านล่าง เพลงนั้นจะถูกโหลดเข้าคิวและเริ่มเล่น
6. ล้างคิว: กดปุ่ม ล้างคิวเพลง ที่มุมขวาบนของกล่องคิว
7. จัดการโปรไฟล์: เปิดหน้า โปรไฟล์ เพื่อแก้ไขชื่อ เบอร์โทรศัพท์ อัปโหลดรูปประจำตัว และเปลี่ยนรหัสผ่าน
8. จัดการโดยผู้ดูแล: ผู้ดูแลเข้า /management/users/ เพื่อเพิ่ม แก้ไข ปิดใช้งาน และลบผู้ใช้ เข้า /management/genres/ และ /management/tags/ เพื่อจัดการแนวเพลงและแท็ก และเปิด /dashboard/ เพื่อดูสถิติและกราฟ

ง. โครงสร้างไฟล์หลักของโครงการ

พาธ	หน้าที่
manage.py	จุดเริ่มต้นของคำสั่งจัดการของ Django
yum_jukebox/settings.py	ค่าตั้งระบบ รวมฐานข้อมูล SECRET_KEY และการตรวจค่าความปลอดภัย
yum_jukebox/urls.py	รวม URL ของโปรเจกต์ รวมถึงเส้นทางเสิร์ฟไฟล์ในโฟลเดอร์ media
music/models.py	โมเดลทั้ง 9 โมเดลและสัญญาณสร้าง Profile อัตโนมัติ
music/forms.py	ฟอร์มสมัครสมาชิก โปรไฟล์ ผู้ใช้ เพลย์ลิสต์ แนวเพลง แท็ก และคิวเพลง
music/views.py	หน้าเครื่องเล่น การค้นหา การจัดการคิว เพลย์ลิสต์ และ API ฝั่งผู้ใช้
music/views_auth.py	สมัครสมาชิก เข้าสู่ระบบ โปรไฟล์ แดชบอร์ด การจัดการผู้ใช้ แนวเพลง แท็ก และเดคคอเรเตอร์ role_required

พาธ	หน้าที
music/urls.py	เส้นทางทั้งหมด 54 เส้นทาง
music/admin.py	การลงทะเบียนโมเดลในหน้าผู้ดูแลของ Django
music/management/commands/ensure_admin.py	คำสั่งสร้างหรือซ่อมบัญชีผู้ดูแลและข้อมูลตัวอย่าง
music/templates/	เทมเพลตทั้งหมด 16 ไฟล์
music/static/	ไฟล์ JavaScript และไฟล์รูปภาพประจำ
music/tests*.py	ชุดทดสอบอัตโนมัติ 25 ไฟล์
music/migrations/	ไฟล์ migration 11 ไฟล์
docs/ios-audio-notes.md	บันทึกการตรวจสอบปัญหาการเล่นเพลงบน iPad และ Safari
Procfile และ Dockerfile	คำสั่งเริ่มระบบสำหรับการนำขึ้นใช้งาน
requirements.txt	รายการแพ็กเกจที่ระบบต้องใช้